

MATHEMATICS FOR ML: MATRICES & APPLICATIONS

Yogesh Haribhau Kulkarni

Outline

1 MATRICES & APPLICATIONS

About Me

Yogesh Haribhau Kulkarni

Bio:

- ▶ 20+ years in CAD/Engineering software development
- ▶ Got Bachelors, Masters and Doctoral degrees in Mechanical Engineering (specialization: Geometric Modeling Algorithms).
- ▶ Currently doing Coaching in fields such as Data Science, Artificial Intelligence Machine-Deep Learning (ML/DL) and Natural Language Processing (NLP).
- ▶ Feel free to follow me at:
 - ▶ Github (github.com/yogeshhk)
 - ▶ LinkedIn (www.linkedin.com/in/yogeshkulkarni/)
 - ▶ Medium (yogeshharibhaukulkarni.medium.com)
 - ▶ Send email to yogeshkulkarni at yahoo dot com



Office Hours:
Saturdays, 2 to 5pm
(IST); Free-Open to all;
email for appointment.

About Content

- ▶ Bare essential mathematics needed for Machine/Deep Learning
- ▶ Just to get you started.
- ▶ Each field within Machine/Deep Learning can go extremely deep.
- ▶ This is to give you enough foundation needed.

About Me

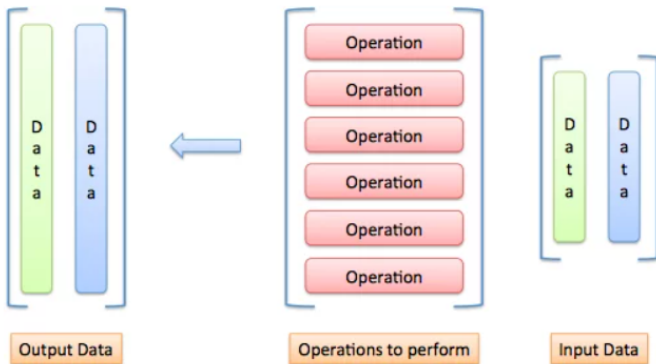
- ▶ I am not a mathematician.
- ▶ And this course won't turn YOU into one!!! (if you are not already)
- ▶ But, I will surely attempt to make you get interested in the learning, more.

Matrix

Meaning of a Matrix

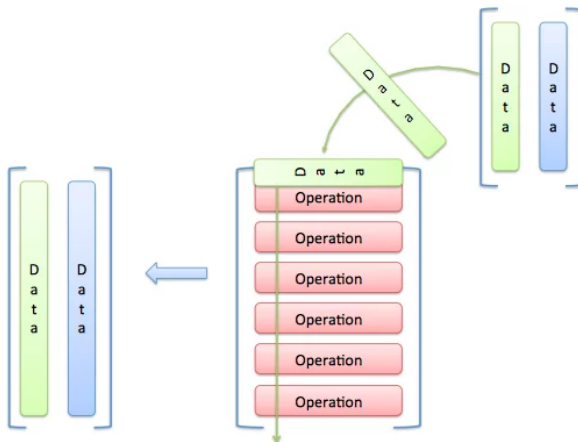
- ▶ Matrix is organization of data into rows and columns
- ▶ Example: columns can be various aspects of a person, such as height, weight, salary, etc, where as rows can represent different persons
- ▶ This Excel sheet like data can be thought of as a Matrix (especially in Data Science, Machine Learning)

Visualizing The Matrix



(Ref: An Intuitive Guide to Linear Algebra - Better Explained)

Visualizing The Matrix Application



(Ref: An Intuitive Guide to Linear Algebra - Better Explained)

Geometric Applications

- ▶ Scale: make all inputs bigger/smaller
- ▶ Skew: make certain inputs bigger/smaller
- ▶ Flip: make inputs negative
- ▶ Rotate: make new coordinates based on old ones (East becomes North, North becomes West, etc.)

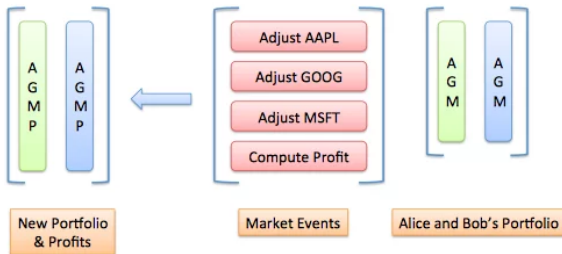
These are geometric interpretations of multiplication, and how to warp a vector space.

(Ref: [An Intuitive Guide to Linear Algebra - Better Explained](#))

Non-Vector Applications

- ▶ Input data: stock portfolios with dollars in Apple, Google and Microsoft stock
- ▶ Operations: the changes in company values after a news event
- ▶ Output: updated portfolios

Linear Algebra (Stock Example)



(Ref: An Intuitive Guide to Linear Algebra - Better Explained)

Solving Simultaneous equations

$$x + 2y + 3z = 3$$

$$2x + 3y + 1z = -10$$

$$5x + -y + 2z = 14$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 5 & -1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 3 \\ -10 \\ 14 \end{bmatrix}$$

You can solve by ...? Some ... Elimination?

(Ref: An Intuitive Guide to Linear Algebra - Better Explained)

Matrix

A matrix is an array of numbers that can be arranged into rows and columns.
We generally name matrices with a capital letter.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

```
1 import numpy as np
3 A = np.array([[1,2,3],
               [4,5,6]])
5 print (A)
7 [[1 2 3]
   [4 5 6]]
```

Matrix

Definition A matrix with m rows and n columns is referred to as an $m \times n$ matrix. The number of rows always comes before the number of columns.

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{bmatrix}$$

```
import numpy as np
2
M = np.matrix([[1,2,3],
4              [4,5,6]])
print (M)
6
[[1 2 3]
8  [4 5 6]]
```

Matrix Addition

You can add or subtract matrices of the same size by simply adding or subtracting the corresponding elements in the two matrices.

$$A = \begin{bmatrix} 3 & 5 & 1 \\ 1 & 4 & 3 \end{bmatrix} \quad B = \begin{bmatrix} 2 & -2 & 4 \\ -1 & 3 & 1 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 5 & 3 & 5 \\ 0 & 7 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 7 & 7 \\ 7 & 7 & 7 \end{bmatrix}$$

Matrix Addition

```
import numpy as np
2
A = np.array([[1,2,3],
4              [4,5,6]])
B = np.array([[6,5,4],
6              [3,2,1]])
print(A + B)
8
[[7 7 7]
10  [7 7 7]]
```

Matrix Subtraction

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} - \begin{bmatrix} 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -5 & -3 & -1 \\ 1 & 3 & 5 \end{bmatrix}$$

```
import numpy as np
2
A = np.array([[1,2,3],
4             [4,5,6]])
B = np.array([[6,5,4],
6             [3,2,1]])
print(A - B)
8
9     [[-5 -3 -1]
10      [ 1  3  5]]
```

The Transpose of a Matrix

Definition The transpose of a $m \times n$ matrix A is the matrix A^T having (i, j) -entry a_{ji} . That is,

$$(A^T)_{ij} = a_{ji}.$$

Example For example, $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ has transpose $A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$.

Note The rows of A become the columns of A^T and vice versa.

Meaning of a Matrix Multiplication

- ▶ Matrix is organization of data into rows and columns
- ▶ Example: columns can be various aspects of a person, such as height, weight, salary, etc, where as rows can represent different persons
- ▶ This Excel sheet like data can be thought of as a Matrix (especially in Data Science, Machine Learning)
- ▶ If you have another matrix like this, what is the meaning of their multiplication?
- ▶ Geometrically: say first matrix represents points of a shape, a polygon, where each row is a point, and each column represents X, Y, Z coordinates.
- ▶ Second matrix is typically a Homogeneous transformation matrix, such as rotation, when multiplied gets rotated shape.

Matrix Multiplication Rules

Theorem Let A and B be matrices whose sizes are appropriate for the following sums and products to be defined

- ▶ $(A^T)^T = A$
- ▶ $(A + B)^T = A^T + B^T$.
- ▶ For any scalar r , $(rA)^T = rA^T$.
- ▶ $(AB)^T = B^T A^T$

Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \text{ and } B = \begin{bmatrix} 5 & 1 & -1 \\ 1 & 2 & 2 \end{bmatrix} \text{ then}$$

$$AB = \begin{bmatrix} 7 & 5 & 3 \\ 9 & 11 & 5 \end{bmatrix} \quad (AB)^T = \begin{bmatrix} 7 & 9 \\ 5 & 11 \\ 3 & 5 \end{bmatrix} = B^T A^T$$

but A^T is 2×2 and B^T is 3×2 , so $A^T B^T$ isn't even defined.

Matrix Transpose

Exchange rows and columns

$$A = \begin{bmatrix} 3 & 5 & 1 \\ 1 & 4 & 3 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 3 & 1 \\ 5 & 4 \\ 1 & 3 \end{bmatrix}$$

Matrix Transpose

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

```
import numpy as np
2
A = np.array([[1,2,3],
4             [4,5,6]])
print(A.T)
6
[[1 4]
8  [2 5]
 [3 6]]
```


Matrix Multiplication

Here are the cases to consider:

- ▶ Scalar multiplication, which is multiplying a matrix by a single number
- ▶ Element wise matrix multiplication (rarely used, called Hadamard multiplication, shown with circle instead of dot)
- ▶ Dot product matrix multiplication, or multiplying a matrix by another matrix.

Matrix Scalar Multiplication

To multiply a matrix by a scalar value, you just multiply each element by the scalar to produce a new matrix:

$$2 \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}$$

```
1 import numpy as np
import numpy as np
3
A = np.array([[1,2,3],
5             [4,5,6]])
print(2 * A)
7
[[ 2  4  6]
9  [ 8 10 12]]
```

Matrix Multiplication Defined

Definition If A is an $m \times n$ matrix, and if $B = [\vec{b}_1, \vec{b}_2 \dots, \vec{b}_p]$ is a $n \times p$ matrix, then the matrix product AB is the following $m \times p$ matrix.

$$AB = [A\vec{b}_1 \quad A\vec{b}_2 \quad \dots \quad A\vec{b}_p]$$

Example Let $A = \begin{bmatrix} 1 & 2 \\ -2 & 3 \end{bmatrix}$ and let $B = \begin{bmatrix} 3 & -1 & 6 \\ 7 & 5 & 3 \end{bmatrix}$. Compute AB .

Multiplying Matrices

Row-Column Rule If A is $m \times n$ and if B is $n \times p$ the (i, j) -entry of AB is given by

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

Note $\text{Row}_i(AB) = \text{Row}_i(A) \cdot B$.

Matrix Operations

Additions

- ▶ Commutative: $A + B = B + A$
- ▶ Associative: $A + (B + C) = (A + B) + C$

Multiplication

- ▶ Scalar : sA : multiplying all elements by s
- ▶ Commutative: $AB \neq BA$
- ▶ Associative: $A(BC) = (AB)C$
- ▶ Distributive: $A(B + C) = AB + AC$
- ▶ Identity: $I_m A_{mn} = A_{mn} I_n = A$

Linear Algebra with Python

(Ref: Linear Algebra and Python Basics - Rob Hicks)

Python Libraries

For numerical computing, useful libraries are:

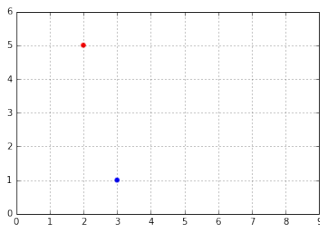
- ▶ `sympy`: provides for symbolic computation (solving algebra problems)
- ▶ `numpy`: provides for linear algebra computations
- ▶ `matplotlib.pyplot`: provides for the ability to graph functions and draw figures
- ▶ `scipy`: scientific python provides a plethora of capabilities
- ▶ `seaborn`: makes `matplotlib` figures even pretties (another library like this is called `bokeh`).

Vectors and Lists

To create a vector simply surround a python list $([1, 2, 3])$ with the *np.array* function:

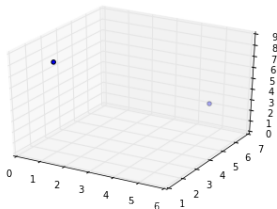
```
1 x_vector = np.array([1,2,3])
  print(x_vector)
3
4 [1 2 3]
5
6 c_list = [1,2]
7 print("The list:",c_list)
  print("Has length:", len(c_list))
9
10 c_vector = np.array(c_list)
11 print("The vector:", c_vector)
  print("Has shape:",c_vector.shape)
13
14 The list: [1, 2]
15 Has length: 2
  The vector: [1 2]
17 Has shape: (2,)
```


2D Vectors



```
1 u = np.array([2, 5])  
  v = np.array([3, 1])  
3  
  x_coords, y_coords = zip(u, v)  
5 plt.scatter(x_coords, y_coords, color=["r", "b"])  
  plt.axis([0, 9, 0, 6])  
7 plt.grid()  
  plt.show()
```

3D Vectors



```
1 a = np.array([1, 2, 8])
2 b = np.array([5, 6, 3])

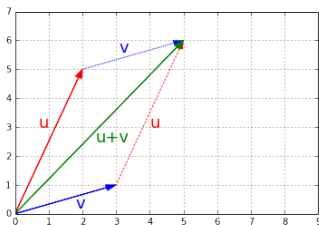
4 from mpl_toolkits.mplot3d import Axes3D

6 subplot3d = plt.subplot(111, projection='3d')
  x_coors, y_coors, z_coors = zip(a,b)
8 subplot3d.scatter(x_coors, y_coors, z_coors)
  subplot3d.set_zlim3d([0, 9])
10 plt.show()
```

vector Norm

```
def vector_norm(vector):  
    squares = [element**2 for element in vector]  
    return sum(squares)**0.5  
  
print(vector_norm(u))  
  
5.3851648071345037  
  
import numpy.linalg as LA  
print(LA.norm(u))  
  
5.3851648071345037
```

Vector Addition



```
1 print(u + v)
3 array([5, 6])
5 plot_vector2d(u, color="r")
5 plot_vector2d(v, color="b")
7 plot_vector2d(v, origin=u, color="b", linestyle="dotted")
7 plot_vector2d(u, origin=v, color="r", linestyle="dotted")
9 plot_vector2d(u+v, color="g")
9 plt.grid()
11 plt.show()
```

Matrices

```
1 b = list(zip(z,c_vector))
  print(b)
3 print("Note that the length of our zipped list is 2 not (2 by 2):",len(b))

5 [(5, 1), (6, 2)]
  Note that the length of our zipped list is 2 not (2 by 2): 2
7
9 D = np.matrix([[1.,2], [3,4], [5,6]])
11 matrix([[ 1.,  2.],
12          [ 3.,  4.],
13          [ 5.,  6.]])
15 E = np.matrix("1.,2; 3,4; 5,6")
17 matrix([[ 1.,  2.],
18          [ 3.,  4.],
19          [ 5.,  6.]])
```

Matrices

```
2 F = np.ones((4,3))
4 array([[ 1.,  1.,  1.],
6        [ 1.,  1.,  1.],
8        [ 1.,  1.,  1.],
10       [ 1.,  1.,  1.]])
8 np.linalg.matrix_rank(F) # np.rank() removed in NumPy 1.25
10 2
```

Matrix Addition and Subtraction

Adding or subtracting a scalar value to a matrix

$$A + 3 = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} + 3 = \begin{bmatrix} a_{11} + 3 & a_{12} + 3 \\ a_{21} + 3 & a_{22} + 3 \end{bmatrix} \quad (1)$$

```
1 result = A + 3 #or result = 3 + A
  print( result)
3
  [[8 4]
  5   [9 5]]
```

Matrix Addition and Subtraction

Adding or subtracting two matrices

$$A_{2 \times 2} + B_{2 \times 2} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}_{2 \times 2} \quad (2)$$

```
1 B = np.random.randn(2,2)
  print( B)
3
  [[-0.9959588   1.11897568]
  [ 0.96218881 -1.10783668]]
5
7 result = A + B
  print(result)
9
  array([[4.0040412 , 2.11897568],
11       [6.96218881, 0.89216332]])
```


Matrix Multiplication

Multiplying a scalar value times a matrix

$$3 \times A = 3 \times \begin{bmatrix} a_{11} & a_{12} & a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} 3a_{11} & 3a_{12} \\ 3a_{21} & 3a_{22} \end{bmatrix} \quad (3)$$

```
1 A * 3
3 array([[15,  3],
        [18,  6]])
```

Matrix Multiplication

Multiplying two matrices

$$A_{3 \times 2} \times C_{2 \times 3} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}_{3 \times 2} \times \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{bmatrix}_{2 \times 3} \quad (4)$$

$$= \begin{bmatrix} a_{11}c_{11} + a_{12}c_{21} & a_{11}c_{12} + a_{12}c_{22} & a_{11}c_{13} + a_{12}c_{23} \\ a_{21}c_{11} + a_{22}c_{21} & a_{21}c_{12} + a_{22}c_{22} & a_{21}c_{13} + a_{22}c_{23} \\ a_{31}c_{11} + a_{32}c_{21} & a_{31}c_{12} + a_{32}c_{22} & a_{31}c_{13} + a_{32}c_{23} \end{bmatrix}_{3 \times 3} \quad (5)$$

```

A = np.arange(6).reshape((3,2))
2 C = np.random.randn(2,2)

4 print( A.dot(C)) # or print( np.dot(A,C))

6 [[-1.19691566  1.08128294]
   [-2.47040472  1.00586034]
8  [-3.74389379  0.93043773]]

```

Matrix Division

A misnomer. To divide in a matrix algebra world we first need to invert the matrix. It is useful to consider the analog case in a scalar work. Suppose we want to divide the f by g . We could do this in two different ways:

$$\frac{f}{g} = f \times g^{-1}. \quad (6)$$

Inverting a Matrix

$$A^{-1} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix} \quad (7)$$

```

C_inverse = np.linalg.inv(C)
2 print( C_inverse)

4 [[-1.47386391 -1.52526704]
   [-1.63147935 -0.76355223]]

```

Matrix Transpose

$$A_{3 \times 2} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}_{3 \times 2} \quad (8)$$

The transpose of A (denoted as A') is

$$A' = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \end{bmatrix}_{2 \times 3} \quad (9)$$

```

1 A = np.arange(6).reshape((3,2))
  print( A)
3 print( A.T)

5 [[0 1]
   [2 3]
   [4 5]]
7 [[0 2 4]
   [1 3 5]]
9

```

Matrix Eigen Values and Vectors

- ▶ Represent the “axes” of the transformation.
- ▶ Consider spinning a globe: every location faces a new direction, except the poles.
- ▶ Along “eigenvector”, when it’s run through the matrix, its points do not rotate (may scale though). The eigenvalue is the scaling factor.

(Ref: <https://en.wikipedia.org/wiki/File:Eigenvectors.gif>)

```
1 from numpy.linalg import eig
  A = np.array([[1,2],[3,4]])
3 eigen_val, eigen_vec = eig(A)

5 print(eigen_val)
  array([-0.37228132,  5.37228132])
7
  print(eigen_vec)
9 array([[ -0.82456484, -0.41597356],
        [ 0.56576746, -0.90937671]])
```

Singular Value Decomposition

Any $m \times n$ matrix M can be decomposed into the dot product of three simple matrices:

- ▶ a rotation matrix U (an $m \times m$ orthogonal matrix)
- ▶ a scaling & projecting matrix Σ (an $m \times n$ diagonal matrix)
- ▶ and another rotation matrix V^T (an $n \times n$ orthogonal matrix)

$$M = U \cdot \Sigma \cdot V^T$$

```
1 U, S_diag, V_T = LA.svd(F)
2
3 print(U)
4 array([[ 0.89442719, -0.4472136 ],
5        [ 0.4472136 ,  0.89442719]])
6
7 print(S_diag)
8 array([ 2. ,  0.5])
```

Linear Algebra - Python Implementation

Vectors

- ▶ If we have data of people with 3 attributes
- ▶ Heights, weights, and ages
- ▶ Each data point is in 3-D space with (height, weight, age) basis.
- ▶ In python, we can use list

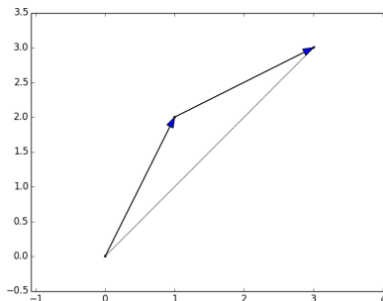
```
2 height_weight_age = [70, # inches,  
                        170, # pounds,  
                        40 ] # years
```


Vectors

- ▶ lists are ok to represent vectors as storage, but not appropriate for operations
- ▶ Why?
- ▶ Whats list additions?
- ▶ Whats vector addition?

Vector Addition

- ▶ We'll frequently need to add two vectors.
- ▶ Vectors add component-wise.
- ▶ Resultant first element is $v[0] + w[0]$,
- ▶ Resultant second element is $v[1] + w[1]$, and so on.
- ▶ If they're not the same length, not allowed to add them.



Vector Addition

- ▶ Implement vector addition
- ▶ Hint: 'zip' two vectors and use List Comprehension

```
1 def vector_add(v, w):  
  :  
3   return [...]  
vv = [ 1, 2,3]  
5 ww = [3,2,1]  
   result = vector_add(vv,ww)  
7 print("Vector Addition {}".format(result))
```

Vector Addition

Solution: Vector Addition [4, 4, 4]

```
1 def vector_add(v, w):  
    """adds corresponding elements"""  
3     return [v_i + w_i  
              for v_i, w_i in zip(v, w)]
```

Vector Subtraction

- ▶ Implement vector subtraction
- ▶ Hint: its an addition with second vector negated

```
def vector_subtract(v, w):  
    :  
    return [...]  
vv = [ 1, 2,3]  
ww = [3,2,1]  
result = vector_subtract(vv,ww)  
print("Vector Subtraction {}".format(result))
```

Vector Subtraction

```
1 def vector_subtract(v, w):  
    """subtracts corresponding elements"""  
3     return [v_i - w_i  
              for v_i, w_i in zip(v, w)]
```

Vectors Summation

- ▶ Implement vector summation
- ▶ Component-wise sum a list of vectors
- ▶ Result is a new vector whose first element is the sum of all the first elements, and so on

```
def vector_sum(vectors):  
    :  
    return [...]  
4 vecs = [[ 1, 2,3],[3,2,1],[3,2,-1]]  
    result = vector_sum(vecs)  
6 print("Vectors Sum {}".format(result))
```

Vectors Summation

```
def vector_sum(vectors):  
    """sums all corresponding elements"""  
    result = vectors[0]  
    for vector in vectors[1:]:  
        result = vector_add(result, vector)  
    return result
```


Scalar Multiplication

- ▶ Implement scalar multiplication of a vector
- ▶ Component-wise multiplication

```
def scalar_multiply(c, v):  
    :  
    return [...]  
vv = [ 1, 2,3]  
cc = 4  
result = scalar_multiply(cc,vv)  
print("Scalar Multiply {}".format(result))
```

Scalar Multiplication

```
1 def scalar_multiply(c, v):  
    """c is a number, v is a vector"""  
3     return [c * v_i for v_i in v]
```

Vectors Mean

- ▶ Component-wise means of a list of (same-sized) vectors:
- ▶ Hint: use `vector_sum` to add all up, then use `scalar_multiply` to compute mean

```
def vector_mean(vectors):  
2     :  
    return [...]  
4 vecs = [[ 1, 2,3],[3,2,1],[3,2,-1]]  
    result = vector_mean(vecs)  
6 print("Vectors Mean {}".format(result))
```

Vectors Mean

Solution: Vectors Mean [2.333333333333333, 2.0, 1.0]

```
def vector_mean(vectors):  
    """compute the vector whose ith element is the mean of the ith elements of  
    the input vectors"""  
    n = len(vectors)  
    return scalar_multiply(1/n, vector_sum(vectors))
```

Vector Multiplication

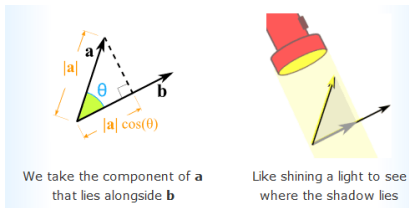
- ▶ Dot Product : Output? Meaning?
- ▶ Cross Product : Output? Meaning?

$$a \cdot b = |a| \times |b| \times \cos(\theta)$$

$$a \times b = |a| \times |b| \times \sin(\theta) \hat{n}$$

Dot Product

- ▶ The Dot Product gives a number as an answer (a “scalar”, not a vector).
- ▶ $a \cdot b = a_x \times b_x + a_y \times b_y$
- ▶ So we multiply the x's, multiply the y's, then add.
- ▶ Can't multiply unless they are in same direction.
- ▶ So, one of them is projected over another using $\cos(\theta)$



If vectors are at right angles? (Reference: <https://www.mathsisfun.com/algebra/vectors-dot-product.html>)

Dot Product

- ▶ The dot product of two vectors is the sum of their component-wise products.
- ▶ Hint: Similar to `vector_add` but with a difference in return type

```
def dot(v, w):  
    :  
    return ...  
vv = [ 1, 2,3]  
ww = [3,2,1]  
result = dot(vv,ww)  
print("Dot Product {}".format(result))
```

Dot Product

```
1 def dot(v, w):
    """v_1 * w_1 + ... + v_n * w_n"""
3     return sum(v_i * w_i
    for v_i, w_i in zip(v, w))
```

Easy to compute a vector's sum of squares:

```
def sum_of_squares(v):
2     """v_1 * v_1 + ... + v_n * v_n"""
    return dot(v, v)
```


Magnitude of a Vector

```
1 import math
2 def magnitude(v):
3     return math.sqrt(sum_of_squares(v))
```

Distance Between Vectors

- ▶ Formula: $\sqrt{(v_1 - w_1)^2 + \dots (v_n - w_n)^2}$
- ▶ Hint: First use `vector_subtract` and then `sum_of_squares`
- ▶ For now, do not bother about normalizing it with product of their magnitudes.

```
def squared_distance(v, w):  
2     :  
    return ...  
4 vv = [ 1, 2,3]  
   ww = [3,2,1]  
6 result = squared_distance(vv,ww)  
   print("Squared Distance {}".format(result))
```

Distance

```
1 def squared_distance(v, w):  
    """(v_1 - w_1) ** 2 + ... + (v_n - w_n) ** 2"""  
3     return sum_of_squares(vector_subtract(v, w))  
  
5 def distance(v, w):  
    return math.sqrt(squared_distance(v, w))  
7  
or  
9  
11 def distance(v, w):  
    return magnitude(vector_subtract(v, w))
```

Matrices

- ▶ A matrix is a two-dimensional collection of numbers.
- ▶ list of lists, with each inner list having the same size and representing a row of the matrix.
- ▶ If A is a matrix, then $A[i][j]$ is the element in the i th row and the j th column.

```
1 A = [[1, 2, 3], # A has 2 rows and 3 columns
      [4, 5, 6]]
3 B = [[1, 2],    # B has 3 rows and 2 columns
      [3, 4],
5      [5, 6]]
```

Matrices

- ▶ Python lists, being '0' indexed, first row of a matrix "row 0" and the first column "column 0".
- ▶ matrix A has $\text{len}(A)$ rows and $\text{len}(A[0])$ columns, which we consider its shape
- ▶ Numpy and Pandas Dataframes have in built matrix functionality needed for Data Science

```
1 def shape(A):  
    num_rows = len(A)  
3    num_cols = len(A[0]) if A else 0  
    return num_rows, num_cols
```

Linear Algebra Summary

(Ref: A Gentle Introduction to Linear Algebra - Json Brownlee)

Summary

- ▶ Linear algebra is about linear combinations.
- ▶ Linear algebra is the study of lines and planes, vector spaces and mappings that are required for linear transforms
- ▶ Applications of Linear Algebra
 - ▶ Matrices in Engineering, such as a line of springs.
 - ▶ Graphs and Networks, such as analyzing networks.
 - ▶ Computer Graphics, such as the various translation, rescaling and rotation of images.

Further Reading

Books:

- ▶ Introduction to Linear Algebra by Serge Lang
- ▶ Introduction to Linear Algebra, Gilbert Strang, 2016.
- ▶ Numerical Linear Algebra, Lloyd N. Trefethen, 1997.
- ▶ Linear Algebra and Matrix Analysis for Statistics, Sudipto Banerjee, Anindya Roy, 2014.

Courses:

- ▶ “Linear Algebra for machine learning” - Patrick van der Smagt
- ▶ “Machine Learning – 03. Linear Algebra Review“. Playlist at Youtube
- ▶ Linear Algebra stream on Khan Academy.

Thanks ...

- ▶ Office Hours: Saturdays, 3 to 5 pm (IST); Free-Open to all; email for appointment to yogeshkulkarni at yahoo dot com
- ▶ Call + 9 1 9 8 9 0 2 5 1 4 0 6



(<https://www.linkedin.com/in/yogeshkulkarni/>)



(<https://medium.com/@yogeshharibhaukulkarni>)



(<https://www.github.com/yogeshhk/>)